DO NOW — RECALL

# Programming Techniques · Scope and Lifetime

Name: _____ Class: _____ Date: _____

**1. What does return do in a method?**

**2. What is a condition-controlled loop?**

**3. Why might binary search outperform linear search?**

**4. Simplify ¬¬B.**

**DO NOW — CHECK / OBJECTIVES**

# Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

### 1. Model answer

Sends a value back to its caller

### 2. Model answer

Repeats while a test holds

### 3. Model answer

It halves the remaining range

### 4. Model answer

B

### By the end, you can…

• Distinguish local and class-level variables

• Trace shadowed identifiers

• Avoid unintended shared state

### Vocabulary — know these terms

Recall the term — scope

Recall the term — local

Recall the term — class field

Recall the term — shadowing

Recall the term — lifetime

NEW LEARNING

# Read, annotate and explain

### Spot two variables with one name

class Program { static int score = 10; static void Main() { int score = 3; System.Console.WriteLine(score); System.Console.WriteLine(Program.score); } }

### Explain the important decision

The class field and method-local variable are different storage locations. C# allows a local to hide a field; this does not permit redeclaring any local in a nested block.

### Explain one step in your own words, using the correct technical terms.

I DO — WORKED EXAMPLE

# Follow the worked method

### Spot two variables with one name

### Identify both variables

Program.score is a class-level static field, initially 10. Main declares its own local score, initially 3.

### Resolve each name

The unqualified score inside Main refers to the local. Program.score explicitly selects the class field; the field has not changed.

### Explain the output and scope

The output is 3 then 10. Another method cannot directly access Main's local score. Restricting scope reduces unintended changes to shared state.

**WE DO — GUIDED PRACTICE**

# Try → Discuss → Improve

### 1. Problem 1

A parameter n is set to 4. What is visible inside the function?

Method / working:

### 2. Problem 2

Why avoid excessive global state?

Method / working:

YOU DO — INDEPENDENT PRACTICE

# Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

### Question 1

Apply independently
Explain one benefit of local variables. [2]

### Question 2

Apply independently
A method declares int total=0. Can main read total directly? [1]

YOU DO — INDEPENDENT PRACTICE

# Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

### Question 3

Apply independently
Define variable shadowing. [2]

**EXIT TICKET / REFLECTION**

# Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

**True or false: every local variable is visible throughout the program.**

**How confident are you? Circle one: Confident / Getting there / Need help**

Explain your choice, identifying one concept you can now use independently.

**GREEN PEN — Correct and explain**

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.